

基本概念：ウィジェットはどのように機能するのか？

Flutter で Key-Value 形式を理解する

a. Key-value 形式は何か？

- Flutter では、ウィジェットのプロパティはキーと値のペアで定義される。
- キー → プロパティを表す（例：style,color）。
- 値 → プロパティの動作を決める（例：Colors.blue,TextStyle）。
- 例として Text ウィジェットに使用：

```
Text(  
  'Hello, Flutter!',      // This is a value (text being  
displayed)  
  style: TextStyle(      // 'style' is a key  
    fontSize: 20,        // 'fontSize' is a key, and '20' is the  
value  
    color: Colors.blue,  // 'color' is a key, and 'Colors.blue'  
is the value  
  ),  
);
```

Hello, Flutter!

b. なぜ Key-value 形式を使用？

- ウィジェットのプロパティをカスタマイズ → UI の外観や動作を簡単に制御できる。
- デフォルト値を上書きできる → 色,サイズ,配置などを自由に変更可能。
- 可読性が向上 → 名前付きパラメータにより,コードが明確になる。

c. いつ key のデフォルト value を使用？

- 値を指定しない場合,Flutter はデフォルト値を適用する。
- 別の動作をさせたいときは,デフォルトを上書きする。
- 例：デフォルトのテキストカラーは黒 → color: Colors.blue に変更可能。
- 例として Text ウィジェットにデフォルトを上書きする：

```
Container(  
  width: 200,  
  height: 100,  
  color: Colors.blue, // This sets the background color to  
blue  
  child: Center(  
    child: Text(  

```

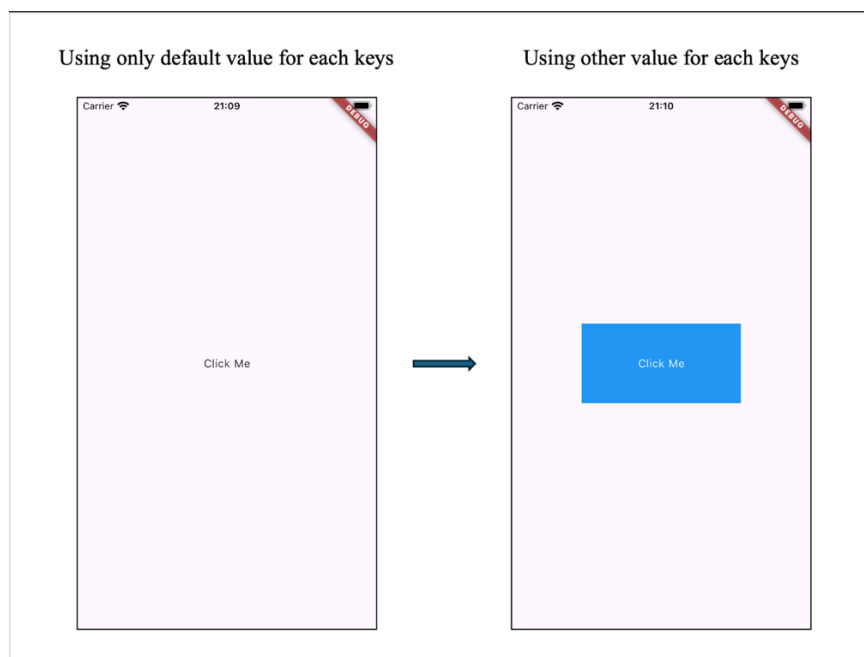
```

        'Click Me',
        style: TextStyle(color: Colors.white), // Text color
        is white
    ),
),
)

```

■ 説明：

- ◇ デフォルトでは,Container に背景色はない.
- ◇ `color: Colors.blue` を設定すると,背景色が青になる.
- ◇ `Text('Click Me')` は中央に配置され,白色で表示される.



d. Key-value 形式を Flutter で使用 vs JSON で使用

- Flutter と JSON はどちらもキーと値のペアを使うが,目的が異なる.
- Flutter → アプリの UI を定義する.

```

Text (
  "Hello, World!",
  textAlign: TextAlign.center,
)

```

- JSON → データの保存や転送を行う.

```

{
  "text": "Hello, World!",
  "textAlign": "center"
}

```

```
}
```

- 主な違い:

特徴	Flutter コード	JSON
目的	UI の構築と動作	データの保存と転送
実行方法	アプリ内で動作	データのやり取りに使用
例	<code>Text('Hello')</code>	<code>{ "text": "Hello" }</code>

Flutter でウィジェットがネストされる理由

a. ウィジェットネストとは？

- ウィジェットのネストとは、あるウィジェットを別のウィジェットの中に配置すること。
- これにより、UI 要素を組み合わせでより複雑なデザインを作成できる。
- Flutter の UI は、ネストされたウィジェットの階層で構成されている。
- 例として `Text` ウィジェットが `Button` ウィジェットの中にネストされる：

```
ElevatedButton(  
  onPressed: () {  
    print('Button clicked!');  
  },  
  child: Text('Click Me'), // Text is inside the Button  
);
```

b. なぜウィジェットネストを使用？

- 関連する UI 要素をまとめる → コードの可読性が向上。
- デザインを柔軟にカスタマイズできる → スタイルやレイアウトの調整が簡単。
- 再利用がしやすい → 繰り返し使えるコンポーネントを作成可能。
- 例として `Text` ウィジェットが `Container` ウィジェットの中にネストされる：

```
Container(  
  padding: EdgeInsets.all(10),  
  child: Text('This is a message'),  
);
```

c. どのようにネストはカスタムに役立つ

- ウィジェットを他のウィジェットの中に配置することで、外観を変更できる。
- 例えば、`Container` を使えばパディング、マージン、背景のスタイルを追加できる。